

Salesforce

Unit	Main Topics
Unit 1	Apex triggers, context variables, helper classes, bulkification and recursion
Unit 2	Asynchronous Apex, Batch Apex, schedulers, monitoring and test classes
Unit 3	Lightning Web Components, LWC bundle, lifecycle, data binding and events
Unit 4	Salesforce integration, REST/SOAP APIs, callouts, Named Credentials and OAuth
Unit 5	Agentforce, Salesforce basics, agent configuration, Omni-Channel and productivity

Rahul yadav

Detailed 5 Unit Semester Exam Notes

1. Understanding Triggers

A trigger in Salesforce is Apex code that automatically executes before or after a record operation. Trigger operations include insert, update, delete, undelete and merge-like data changes. Triggers are used when declarative automation is not enough or when complex business logic is required.

Before triggers are used to validate or modify values before a record is saved to the database. After triggers are used when

- Before insert/update: used for field validation and changing current record values.
- After insert/update/delete: used for related record changes, callouts through async methods and custom logic.
- Triggers run automatically and do not need to be called manually.
- Poorly written triggers can cause governor limit errors and recursion.

Trigger Event	When It Runs	Common Use
before insert	Before new record is saved	Set default field values
before update	Before edited record is saved	Validate or modify fields
before delete	Before record deletion	Prevent deletion
after insert	After new record is saved	Create related records
after update	After edited record is saved	Update child/parent records
after delete	After record deletion	Cleanup related records
after undelete	After record restoration	Restore related data

Salesforce provides trigger context variables to identify which operation is running and what data is available. These variables help developers write logic for insert, update, delete and undelete events. For example, `Trigger.new` contains new versions of records, while `Trigger.old` contains old versions.

Trigger context variables are very important in exams and interviews because they decide whether data can be modified

- `Trigger.isBefore` and `Trigger.isAfter` identify timing.
- `Trigger.isInsert`, `isUpdate`, `isDelete` and `isUndelete` identify operation.
- `Trigger.new` contains new record list for insert/update/undelete.

record Id is available and when related records need to be created or updated. A good trigger should be bulkified, simple and should call a helper class for business logic.

2. Trigger Context Variables

directly or must be updated using DML.

- Trigger.old contains old record list for update/delete.
- Trigger.newMap and Trigger.oldMap provide Id-to-record maps.
- Trigger.size gives number of records in current trigger execution.

3. Basic Trigger Syntax

```
trigger AccountTrigger on Account (before insert, before update)
{
    for (Account acc : Trigger.new) {
        if (String.isBlank(acc.Description)) {
            acc.Description = 'Created or updated by trigger';
        }
    }
}
```

In the above example, the trigger runs before Account insert and update. Because it is a before trigger, fields of Trigger.new records can be changed directly without writing an update DML statement.

4. Trigger Helper Class Best Practice

A recommended trigger design is to keep trigger code short and move logic into a helper or handler class. This improves readability, testing and maintenance. A trigger should mainly detect the event and call the correct helper method. Helper classes are useful when a trigger has many operations such as before insert, after update and after delete. It also

makes recursion control and bulk processing easier.

- Use static helper methods for event-specific logic.
- Make code bulk-safe by processing lists and maps.

```
trigger AccountTrigger on Account (before insert, before update) {  
    AccountTriggerHandler.setDefaultRating(Trigger.new);  
}
```

```
public class AccountTriggerHandler {  
    public static void setDefaultRating(List<Account> accounts) {
```

5. Bulkifying and Avoiding Recursive Triggers

Bulkification means writing Apex code that works correctly for one record as well as many records. Salesforce triggers can execute for up to 200 records in one transaction. If SOQL or DML is written inside loops, governor limits may be exceeded.

Recursive triggers occur when trigger logic updates the same object again, causing the trigger to run repeatedly. Static Boolean flags or better logic design can prevent recursion.

- Use one SOQL query to fetch all needed related records.
- Use static variables to control recursion when required.
- One trigger per object is recommended.
- Keep trigger logic outside trigger body.

```
if (Trigger.isBefore) {  
    }  
}  
  
for (Account acc : accounts) {  
    if (String.isBlank(acc.Rating)) {  
        acc.Rating = 'Warm';  
    }  
}
```

- Do not write SOQL queries inside loops.
- Do not write DML operations inside loops.
- Use List, Set and Map collections.

```
public class TriggerControl {    public
static Boolean isFirstRun = true; }

trigger AccountTrigger on Account (after update) {
if (TriggerControl.isFirstRun) {
TriggerControl.isFirstRun = false;
    // safe logic here
}
}
```

Hands-on idea: Create a trigger that updates Account Description when Rating changes, and test it with multiple Account records at once.

1. Asynchronous Processing and Batch Apex

Asynchronous Apex is used to run processes in the background. It is useful for long-running tasks, large data operations, callouts and scheduled processing. Unlike synchronous execution, asynchronous jobs do not block the user interface.

Batch Apex is used to process large numbers of records by splitting them into small batches. It is very useful when millions of records need updates, data cleanup or periodic processing. Batch Apex implements Database.Batchable interface.

- Handles large data volumes safely.
- Each batch gets separate governor limits.
- Can be monitored from Apex Jobs.
- Supports schedule-based execution.
- Useful for data migration, cleanup and mass updates.



2. Batch Apex Components

A Batch Apex class contains three main methods: start, execute and finish. The start method returns records to process. The execute method processes each batch. The finish method runs after all batches are completed.

Method	Purpose
start()	Collects records using Database.QueryLocator or Iterable.
execute()	Processes each group of records.
finish()	Runs final logic such as email notification or chaining jobs.
Database.QueryLocator	Used for SOQL query and can handle large records.
Iterable	Used when records come from custom collection logic.

```

global class AccountBatch implements Database.Batchable<SObject> {
    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator(
            'SELECT Id, Description FROM
            Account WHERE Description = NULL'
        );
    }

    global void execute(Database.BatchableContext bc, List<Account> scope)
    {
        for (Account acc : scope) {
            acc.Description =
            'Updated by Batch Apex';
        }
        update scope;
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Batch completed successfully');
    }
}
  
```

3. Scheduling and Monitoring Batch Jobs

Batch jobs can be executed immediately using `Database.executeBatch()` or scheduled for a specific time using Apex

Batch jobs can be monitored from Setup by searching Apex Jobs. Developers can check status, number of processed batches, errors and completion time. Debug logs help identify failures.

- Implement `Schedulable` interface for scheduler class.
- Use debug logs and `AsyncApexJob` object for troubleshooting.

```
global class AccountBatchScheduler implements Schedulable {  
    global void execute(SchedulableContext sc) {  
        Database.executeBatch(new AccountBatch(), 200);  
    }  
}
```

```
System.schedule('Daily Account Batch', cronExp, new AccountBatchScheduler());
```

4. Salesforce Testing and Test Classes

Test classes are used to verify Apex code behavior. Salesforce requires at least 75 percent Apex code coverage for deployment to production. However, the main purpose of testing is not only coverage but also correctness and reliability.

Test methods create test data, execute code, and verify results using `System.assert` methods. Test data created in tests is not saved permanently. The `@isTest` notation marks classes and methods as test-only.

- Use `Test.startTest()` and `Test.stopTest()` to reset limits and run async code.
- Avoid relying on existing org data; create your own test data.

Scheduler. Scheduling is useful for nightly data cleanup, weekly summary updates or periodic integration jobs.

- Use `System.schedule()` to schedule a job.
- Monitor jobs from Setup -> Apex Jobs.

```
}  
}  
  
// Run every day at 1 AM
```

```
String cronExp = '0 0 1 * * ?';
```

- Use `@isTest` annotation for test class.
- Use `@testSetup` to create reusable test data.
- Use assertions to verify expected behavior.

```
@isTest private class AccountTriggerTest {
    @testSetup    static void setupData() {
        insert new Account(Name = 'Test Account');    }

    @isTest
    static void testDefaultRating() {                Account acc =
        [SELECT Id, Rating FROM Account LIMIT 1];      acc.Name =
        'Updated Account';

        Test.startTest();
        update acc;
        Test.stopTest();

        Account result = [SELECT Rating FROM Account WHERE Id = :acc.Id];
        System.assertEquals('Warm', result.Rating);
    }
}
```

1. Overview and Evolution of Lightning Framework

Lightning Web Components are Salesforce's modern UI framework based on standard web technologies such as HTML, JavaScript and CSS. LWC was introduced after Aura Components to provide better performance, smaller code size and easier development.

Aura is Salesforce's older component model. LWC uses browser standards and modern JavaScript, so it is faster and easier

- It supports reusable component-based development.
- It works with Apex, Lightning Data Service and Salesforce UI APIs.

Point	Aura	LWC
Technology	Salesforce-specific component model	Standard HTML, CSS and JavaScript
Performance	Comparatively slower	Faster and lightweight
Learning	Needs Aura-specific syntax	Uses modern web standards
Development	Older framework	Recommended modern framework

An LWC component is stored as a bundle. The bundle contains HTML, JavaScript, XML configuration and optionally CSS. The HTML file defines UI structure, JavaScript handles logic and data, CSS handles styling, and XML controls where the

Salesforce DX and scratch orgs are used for modern development. Developers write code locally in VS Code, push source to

- .js file: JavaScript class, properties and methods.
- Salesforce DX helps with source-driven development.

for web developers to learn. LWC components can be used in Lightning App Builder, record pages, app pages and Experience Cloud.

- LWC is lightweight and standards-based.
- It improves performance compared to Aura.

2. Anatomy of an LWC

component is available.

org, test component and deploy it.

- .html file: template and markup.
- .css file: component-level styling.
- .js-meta.xml file: metadata configuration.

```
<!-- helloWorld.html -->
<template>
  <lightning-card title="Hello LWC">
    <p class="slds-p-around_medium">Hello, {studentName}</p>
    <lightning-button label="Change" onclick={handleChange}></lightning-button>
  </lightning-card>
</template>

// helloWorld.js
import { LightningElement } from 'lwc';

export default class HelloWorld extends LightningElement {
  studentName = 'Rahul';

  handleChange() {
    this.studentName =
'Salesforce Student';
  }
}

<!-- helloWorld.js-meta.xml -->
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>60.0</apiVersion>
  <isExposed>true</isExposed>
  <targets>
    <target>lightning__AppPage</target>
    <target>lightning__RecordPage</target>
    <target>lightning__HomePage</target>
  </targets>
</LightningComponentBundle>
```

3. Core Concepts: Lifecycle, Data Binding and Events

LWC lifecycle hooks are special methods that run at different stages of a component's life. constructor runs when the component is created. connectedCallback runs when component is inserted into DOM. renderedCallback runs after rendering. disconnectedCallback runs when component is removed.

Data binding connects JavaScript properties with HTML template. If a property changes, the UI updates automatically. Events are used for communication between components or between user actions and component logic.

- connectedCallback(): component added to DOM.

- CustomEvent is used for child-to-parent communication.

```
const selectedEvent = new CustomEvent('selected', {  
  detail: { value: 'Account Selected' }  
});
```

Hands-on idea: Create an LWC that displays account name, accepts user input and fires a custom event on button click.

- constructor(): component creation.

- renderedCallback(): after rendering.

- disconnectedCallback(): component removed.

- onclick and onchange handle user events.

```
// child component event example  
handleClick() {  
  
  });  
  this.dispatchEvent(selectedEvent);  
}
```

1. Introduction to Salesforce Integration

Salesforce integration connects Salesforce with external systems such as payment gateways, ERP, websites, mobile apps or other cloud platforms. Integration helps exchange data and automate business processes across systems.

Salesforce supports multiple APIs. REST API is lightweight and commonly used for web and mobile applications. SOAP API is XML-based and often used in enterprise integrations. Bulk API is used for large data operations.

- Inbound means external system calls Salesforce.
- Outbound means Salesforce calls external system.
- REST uses HTTP methods such as GET, POST, PUT and DELETE.

API	Format	Use
REST API	JSON/XML over HTTP	Lightweight web/mobile integrations
SOAP API	XML	Enterprise integrations with strict contracts
Bulk API	CSV/JSON batches	Large data load and export
Streaming API	Events	Real-time notifications
Metadata API	Metadata XML	Deploy, configuration and metadata

2. Outbound Integrations and Named Credentials

Outbound integration means Salesforce sending a request to an external system. In Apex, callouts are made using `HttpRequest`, `Http` and `HttpResponse` classes. Salesforce does not allow callouts after DML in the same transaction unless

Named Credentials store endpoint URL and authentication details securely. They reduce hardcoding of credentials and make

- Use Named Credential instead of hardcoded endpoint and token.
- Use asynchronous Apex for long-running callouts.
- Integration can be inbound or outbound. - SOAP uses XML messages and WSDL.

async design is used carefully.

integration safer and easier to maintain.

- Use `HttpRequest` to create request.
- Use GET to fetch data and POST to send data.

```
public class WeatherService {    public static void getWeather() {  
    HttpRequest req = new HttpRequest();  
    req.setEndpoint('callout:Weather_Named_Credential/current');  
    req.setMethod('GET');  
  
        Http http = new Http();  
        HttpResponse res = http.send(req);  
        System.debug(res.getBody());  
    }  
}
```

3. Inbound Integrations and RESTful Services

Inbound integration means an external system sends requests to Salesforce. Apex REST classes can expose custom endpoints. The `@RestResource` annotation defines the endpoint URL mapping. Methods such as `@HttpGet` and `@HttpPost` define supported HTTP operations.

Authentication is required for secure inbound integration. OAuth 2.0 is commonly used to authorize external apps. JWT flow can be used for server-to-server integration where user interaction is not required.

- Use `@RestResource` to expose Apex as REST service.
- Use `@HttpGet` to return data.
- Use `@HttpPost` to create data.
- OAuth 2.0 provides secure access tokens.

- JWT is useful for backend system authentication.

```
String accId = req.requestURI.substring(req.requestURI.lastIndexOf('/') + 1);  
return [SELECT Id, Name, Industry FROM Account WHERE Id = :accId LIMIT 1];
```

Postman is used to test APIs by sending HTTP requests and checking responses. It is useful for testing OAuth tokens, REST endpoints and request bodies. Salesforce Workbench is a browser-based tool used to explore Salesforce APIs, run SOQL

These tools are important in hands-on practice because they help debug endpoint URL, authentication, request body and response format before writing full application code.

- Debug logs help trace Apex callout and HTTP service execution.
- Named Credentials improve security in outbound integrations.

```
@RestResource(urlMapping='/AccountService/*')  
global with sharing class AccountRestService {  
    @HttpGet  
    global static Account getAccount() {  
        RestRequest req = RestContext.request;  
  
    }  
}
```

4. Integration Tools

queries and test REST calls.

- Postman tests REST API endpoints.
- Workbench explores Salesforce data and APIs.

1. Understanding Agentforce

Agentforce is Salesforce's AI-powered agent capability that helps businesses automate service, sales and operational tasks. It can assist users, answer questions, perform actions and improve productivity inside Salesforce workflows.

Agentforce is useful for customer support, internal helpdesk, sales assistance and guided workflows. It can work with Salesforce data, automation and communication channels to support users and agents.

- Can assist agents with suggested responses and actions.
- Useful in service, sales and customer engagement use cases.

Salesforce is a cloud-based CRM platform used to manage customers, sales, service, marketing and business processes. It provides standard applications and allows customization through objects, fields, flows, Apex and Lightning components.

Salesforce data is stored in objects. Standard objects are provided by Salesforce, such as Account, Contact, Lead, Opportunity and Case. Custom objects are created by users to store business-specific data.

- Custom object: user-created object ending with __c.

Concept	Meaning	Example
App	Group of tabs and features	Sales App, Service App
Object	Stores business data	Account, Contact, Student__c
Field	Stores one type of value	Name, Phone, Status__c
Record	One data entry	One Account record
Profile	Controls permissions	System Administrator
Role	Controls data visibility hierarchy	Sales Manager

- Automates repetitive support and service tasks.
- Uses Salesforce data and business context.
- Improves response speed and productivity.

2. Salesforce Platform Basics

- App: collection of tabs and functionality.
- Object: table-like structure to store records.
- Field: column-like data attribute.
- Record: single row of object data.
- Standard object: built-in Salesforce object.

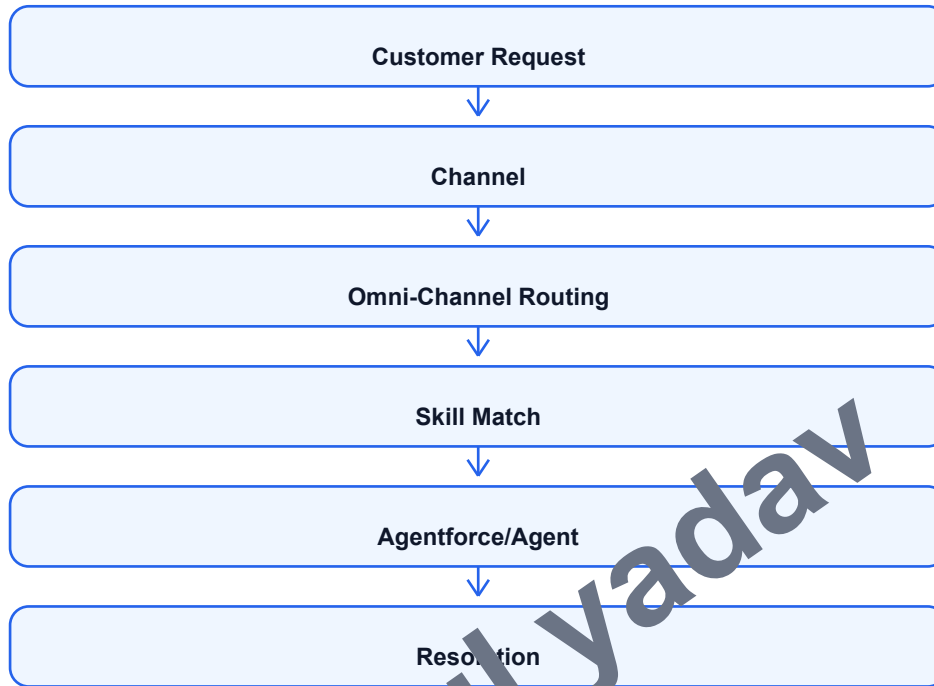
3. Configuring Agentforce

Configuring Agentforce involves defining agent profiles, roles, communication channels, skills and availability. A profile controls what the agent can access. A role helps define visibility and hierarchy. Skills help route work to the correct agent or AI capability.

Communication channels may include chat, messaging, email or voice depending on business requirements. Agent availability ensures that work is assigned only when agents are ready. Skills-based routing improves customer experience by sending work to the best-suited agent.

- Create custom agent profiles and permissions.
- Define agent roles for access and reporting.
- Configure communication channels such as chat or messaging.
- Set agent availability status.
- Define skills for routing work.

- Test routing and agent actions before production use.



4. Omni-Channel and Productivity

Omni-Channel is a Salesforce feature that routes work items to the right agents based on availability, capacity and skills. It supports cases, chats, leads and custom objects. It helps distribute work automatically instead of manually assigning records.

Productivity increases because agents receive the right work at the right time. Supervisors can monitor queues, workload and performance. When combined with Agentforce, agents can receive AI assistance, suggestions and faster responses.

- Routes work based on priority, capacity and skills.
- Reduces manual assignment effort.
- Improves response time and service quality.
- Supports supervisor monitoring.
- Works well with AI assistance and automation.

Simple Apex Example: Create Case Automatically

```

public class CaseCreator {
    public static Case createSupportCase(String subjectText) {
        Case c = new Case();
        c.Subject = subjectText;
        c.Status = 'New';
        c.Origin =
'Web';
        insert c;
        return c;
    }
}
  
```

Unit 1

- What are triggers? Explain before and after triggers.
- Explain trigger context variables.
- Write syntax of a basic Apex trigger.
- Explain trigger helper class and best practices.

- What is trigger bulkification? How can recursive triggers be avoided?

- Explain start, execute and finish methods of Batch Apex.

- How can batch jobs be scheduled and monitored?

- Explain test classes, @isTest and @testSetup with example.

- Explain lifecycle hooks, data binding and events in LWC.

- Explain Salesforce integration and types of APIs.

- Explain OAuth 2.0, JWT, Postman and Workbench.

- Explain Salesforce platform, objects, fields and records.

- Explain Agentforce and its role in Salesforce.

- Differentiate standard and custom objects.

- Explain Agentforce configuration.

- Explain Omni-Channel and productivity features.

Unit 2

- Explain asynchronous Apex and Batch Apex.

- Write a Batch Apex class example.

Unit 3

- What is LWC? Explain benefits of LWC.

- Compare LWC and Aura Components.

- Explain anatomy of an LWC bundle.

- Write a simple LWC example.

Unit 4

- Differentiate REST and SOAP API.

- Explain Apex callouts and Named Credentials.

- Write Apex REST service example.

Unit 5